

trj_analysis: GPU-accelerated analysis of large LAMMPS NetCDF trajectories

Enrique Lomba^{a,b,*}, Rodrigo Lomba-Moreno^c, Antonio Díaz-Pozuelo^{a,**}

^a*Instituto de Química Física Blas Cabrera, CSIC, Madrid, Spain*

^b*Grupo NAFOMAT, Facultade de Física, Universidade de Santiago de Compostela, Santiago de Compostela, Spain*

^c*ETSI Ingeniería Informática, Campus de Montegancedo, Universidad Politécnica de Madrid, Madrid, Spain*

Abstract

We present `trj_analysis`, a high-performance trajectory analysis toolkit for molecular dynamics simulations generated with LAMMPS and stored in NetCDF format. The code combines modern Fortran (F90/F2008) with GPU acceleration via NVIDIA compilers (`nvfortran` and `nvcc`). The package implements a range of structural and dynamical observables, including longitudinal and transverse current correlations and their spectra, optimized for large systems and long trajectories. A modular architecture supports extensibility, offering comprehensive tools for density-based cluster identification, charge-charge correlations, geometric cluster-surface identification, and confinement-resolved structural profiles. Performance benchmarks illustrate significant speedups using GPU acceleration. We include validation against reference data and provide examples and documentation to support reproducible workflows.

Keywords: Molecular dynamics, Trajectory analysis, LAMMPS, NetCDF, GPU acceleration, CUDA Fortran

Program Summary

Program Title: `trj_analysis`

CPC Library link: (to be assigned)

Developer's repository link: https://github.com/elomba/trj_analysis

Licensing provisions: GPL V3.0

Programming language: Fortran (F90/F2008), C++

External routines/libraries: NetCDF (C and Fortran), FFTW3, CUDA toolkit, BLAS and LAPACK

Nature of problem: Analysis of large LAMMPS trajectories stored in NetCDF format[1] can be time-consuming for high-resolution structural and dynamical quantities, especially when

*Co-corresponding author.

**Corresponding author.

Email addresses: enrique.lomba@csic.es (Enrique Lomba), adiaz@iqf.csic.es (Antonio Díaz-Pozuelo)

cluster analysis is also required. Efficient, scalable tools are needed to process data for scientific insight.

Solution method: The program `trj_analysis` provides a modular GPU-accelerated framework for trajectory post-processing based on NetCDF trajectory files generated by LAMMPS. The code implements parallel algorithms for radial distribution functions, static and dynamic structure factors, cluster identification, confinement-resolved density profiles, time correlation functions, longitudinal and transverse current spectra, and Steinhardt bond-orientational order parameters. Independent analysis modules are activated through a structured namelist input interface, allowing flexible workflows adapted to heterogeneous systems and large datasets.

Restrictions: The program supports both single-component and multi-component systems in two and three spatial dimensions. Confinement analysis is restricted to a slit geometry along the z-axis for 3D systems. Input trajectories must be provided in NetCDF format. GPU acceleration requires NVIDIA hardware and corresponding toolchains. The code assumes orthogonal simulation cells and a constant number of particles. Currently only real, metal, and lj units are supported. This code produces atom-oriented observables, molecular properties are not computed at this stage, but an analysis of topology files is provided to generate information on molecular species present in the simulation sample, and to perform a proper calculation of degrees of freedom in systems with rigid fragments.

Additional comments: Examples, validation routines and pre- and postprocessing tools are distributed with the software. **Running time:** Execution time depends on system size, trajectory length, and selected analysis modules. Typical runs for trajectories containing 10^4 – 10^5 particles and 10^3 configurations complete within minutes on a modern GPU-enabled workstation.

1. Introduction

Molecular dynamics (MD) simulation is a cornerstone of computational studies in condensed matter, soft matter, and biophysics. State-of-the-art simulation packages such as LAMMPS[2] can generate trajectories that span millions of particles and millions of time steps, producing datasets whose analysis alone poses significant computational challenges. While MD engines provide efficient integration and sampling, post-processing tools must be equally robust to support modern workflows.

`trj_analysis` addresses this issue by offering a high-performance, modular trajectory analysis toolkit for LAMMPS NetCDF dumps[1]. It supports GPU acceleration to reduce the wall-clock time for computationally intensive routines and provides a suite of observables commonly required in physics and materials science studies. The code is implemented in modern Fortran and C++, and it is openly available under a GPL V3.0 license.

It is worth noticing that several general-purpose trajectory analysis frameworks are available for molecular dynamics simulations. Among the most widely used are MDAnalysis [3, 4] and MDTraj [5], which provide flexible Python-based environments supporting multiple trajectory formats and a broad range of structural observables. These packages are particularly well suited for biomolecular simulations and interactive analysis workflows, but their reliance on interpreted scripting layers and in-memory trajectory representations can limit performance when processing very large particle-based datasets.

For soft-matter and particle-based simulations, the `freud` library [6] offers efficient implementations of neighbor finding, structural correlations, clustering tools, and bond-orientational order parameters through a C++ backend with Python bindings. These capabilities make it a widely used platform for structural characterization in simulations performed with packages such as HOOMD-blue [7] and LAMMPS. However, its execution model remains primarily CPU-based and is typically integrated into Python workflows rather than standalone high-throughput analysis pipelines.

Compiled trajectory-analysis toolkits such as L00S [8] and Pteros [9] provide efficient C++ interfaces for trajectory manipulation and structural analysis. While these packages offer improved performance compared with interpreted environments, they do not currently include GPU-accelerated implementations of histogram-based observables or density-based clustering algorithms tailored for large-scale particle simulations. Visualization-oriented environments such as OVITO [10] provide advanced structural characterization tools, including common-neighbor analysis and bond-orientational order parameters, together with scripting interfaces for automated workflows. These tools are primarily designed for interactive inspection and post-processing rather than high-throughput batch evaluation of long trajectories.

In contrast to these packages, `trj_analysis` is specifically designed for scalable post-processing of large LAMMPS trajectories stored in NetCDF format. The combination of compiled Fortran/C++ kernels with GPU acceleration enables efficient evaluation of histogram-based structural observables, bond-orientational order parameters, and density-based cluster identification for systems containing hundreds of thousands of particles over long simulation times within a unified command-line workflow.

2. Software Design and Implementation

The software described here is version 1.7.6 (4 September 2026), repository revision 3d9cc1f. Relative to the version available at submission on 8 May 2026, it adds current correlations and spectra, optional processing of non-decaying intermediate scattering functions, and geometric cluster-surface detection. The revision also corrects radial binning, bond-order evaluation, Fourier-spectrum output, and stress-correlation initialization and unit conversion.

2.1. Objectives

The primary design goals are:

- Efficient input of NetCDF trajectory files.
- Scalable computation of structural and dynamical observables.
- Modular separation of I/O, data structures, and analysis kernels.
- GPU acceleration for performance-critical operations.

2.2. Architecture

The software consists of:

- A core I/O layer for reading NetCDF trajectories.
- A data model representing per-frame particle data and simulation metadata.

- Analysis modules implementing specific observables.
- Backend kernels for CPU and GPU execution.

Fortran (F90/F2008) is used for the main application logic due to its strong support for numerical arrays, while C++ provides interface to CUDA Thrust library. GPU acceleration is enabled through NVIDIA's `nvfortran` and `nvcc` compilers which incorporate the required CUDA language extension to both Fortran and C++.

3. Analysis workflow

The analysis pipeline implemented in `trj_analysis` is illustrated in Fig. 1. If needed, simulation trajectories generated with LAMMPS in the standard text-based dump format can be preprocessed using utilities included in the `tools` directory and transformed to the appropriate NetCDF format[1] before being analysed by the modular routines that compute structural, cluster and dynamical observables.

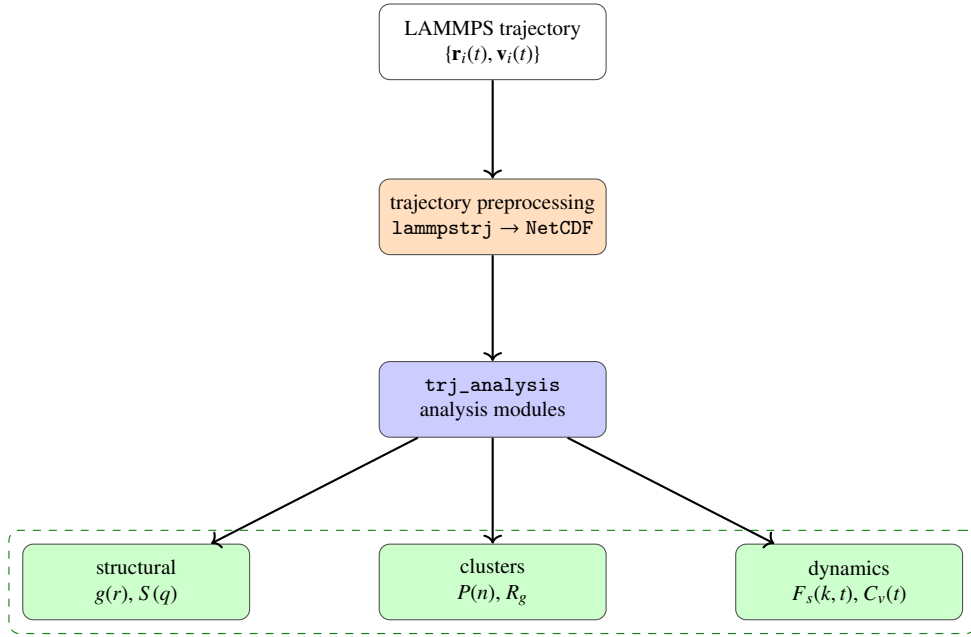


Figure 1: Workflow of the `trj_analysis` package.

4. Supported Observables

The current implementation includes the following analyses integrated in the codebase:

4.1. Structural Analysis

4.1.1. Pair distribution functions

The radial distribution function is defined as

$$g(r) = \frac{1}{4\pi r^2 \rho N} \left\langle \sum_{i,j} \delta(r - r_{ij}) \right\rangle, \quad (1)$$

For a system of N particles in a volume V with number density $\rho = N/V$, the discretized radial distribution function is computed as

$$g(r_k) = \frac{1}{N\rho 4\pi r_k^2 \Delta r} \left\langle \sum_{i=1}^N \sum_{j \neq i} \Theta(r_{ij} - r_k + \frac{\Delta r}{2}) \Theta(r_k + \frac{\Delta r}{2} - r_{ij}) \right\rangle, \quad (2)$$

where $r_{ij} = |\mathbf{r}_i - \mathbf{r}_j|$, Δr is the bin width, and Θ is the Heaviside function. This translates into

$$g(r_k) = \frac{H(r_k)}{N\rho \Delta V_k}, \quad (3)$$

where $H(r_k)$ counts ordered pairs in the bin. The implementation counts each unordered pair once and includes the corresponding factor of two in the normalization. Bins have edges $(k-1)\Delta r$ and $k\Delta r$, centers $r_k = (k - \frac{1}{2})\Delta r$, and exact shell volumes $\Delta V_k = 4\pi[(k\Delta r)^3 - ((k-1)\Delta r)^3]/3$ in three dimensions. In two dimensions, the corresponding annular area is $\pi[(k\Delta r)^2 - ((k-1)\Delta r)^2]$. This consistent binning replaces the thin-shell approximation in the numerical normalization. Our current implementation makes use of tiling approaches to take advantage of fast shared memory access in the GPU for the computation of the particle-particle histograms following closely the strategy of Reuter and Köfinger[11]. The code supports species-resolved correlations $g_{\alpha\beta}(r)$ as well as explicit charge-charge radial distribution functions $g_{qq}(r)$ if partial charges are provided in the trajectory dump. Results are stored in `gmixsim.dat`. Here it is worth to stress a particular feature of the program's internal logic. For efficiency, LAMMPS atom types are remapped into contiguous internal IDs, $1 \dots \text{nsp}$ (where `nsp` is the number of atomic species), and then particle positions are also remapped into the system memory once a simulation frame is read, so as to keep all particles of the same species in contiguous memory locations. Molecule IDs, when present, are mapped consistently with the selected atoms and retained in configuration exports; the observables remain atom-based. This minimizes non-contiguous memory accesses when computing species-resolved properties in mixtures.

4.1.2. Particle Number Fluctuations and Hyperuniformity

In conjunction with the calculation of the radial distribution functions, this code provides the possibility to characterize the large-scale density fluctuations of point patterns so as to assess the presence of hyperuniformity[12]. To that aim we analyze the number variance $\sigma_N^2(R)$ within a spherical observation window Ω of radius R . The number of particles $N(R)$ in the window centered at position $\mathbf{x}_0$ is defined as:

$$N(R; \mathbf{x}_0) = \sum_{i=1}^N \chi_R(\mathbf{r}_i - \mathbf{x}_0) \quad (4)$$

where $\chi_R(\mathbf{r})$ is the indicator function of the window, which is 1 if $|\mathbf{r}| \leq R$ and 0 otherwise. The local number variance is then:

$$\sigma_N^2(R) = \langle N(R)^2 \rangle - \langle N(R) \rangle^2 \quad (5)$$

where $\langle \dots \rangle$ denotes an ensemble average or an average over window centers $\mathbf{x}_0$. This quantity is computed both for particles and for cluster centers if the cluster analysis is enabled and the number of clusters is larger than `clthresh`. The number of sampling origins is controlled by the input variable `nrandom`. When this is set to zero (default value) no fluctuation number analysis is carried out.

4.1.3. Static structure factors $S(q)$

This quantity is defined as

$$S(q) = \frac{1}{N} \langle \rho(\mathbf{q}) \rho(-\mathbf{q}) \rangle. \quad (6)$$

which once discretized reads

$$S(\mathbf{q}) = \frac{1}{N} \left\langle \left| \sum_{j=1}^N e^{i\mathbf{q} \cdot \mathbf{r}_j} \right|^2 \right\rangle. \quad (7)$$

Note that this quantity can also be computed from the Fourier Transform of the radial distribution function. The direct evaluation approach produces more accurate results for low- Q values, which would otherwise be affected by truncation errors in $g(r)$. Additionally, finite-size effects in $g(r)$ are known to be more pronounced in the canonical ensemble than in the grand-canonical ensemble, where particle-number fluctuations allow a faster convergence towards the thermodynamic limit. In particular, the pair distribution functions exhibit corrections of order $1/V$ in the canonical ensemble, while grand-canonical estimates are generally closer to their bulk values [13, 14, 15].

Here the parallelization has been straightforward, with each GPU thread being responsible to compute the contribution of each $\mathbf{Q}(ijk) = 2\pi(i/L_x, j/L_y, k/L_z)$, with $i, j, k = 0, \dots, n_{max}$ and L_m being the simulation cell dimension along the corresponding m -axis. Note that the periodicity of the simulation box constraints the values of the $\mathbf{Q}$ vector. The current program version only gives meaningful structure factors (either static or dynamic) for fully periodic samples. All contributions are added up in bins up to `qmin`, and from `qmin` to `qmax` only the $\mathbf{Q}$ -vectors $2\pi(i/L_x, 0, 0)$, $2\pi(0, j/L_y, 0)$, and $(0, 0, k/L_z)$ are averaged. Results for partial structure factors are stored in `sqmix.dat`, and total structure factors in `sq.dat`.

4.1.4. Confinement and Directional Profiles

For systems under slit-pore confinement along the z -axis, the toolkit extracts highly localized structural information:

- **Density Profiles:** 1D number density and charge density profiles computed across the z -axis ($\rho(z)$ and $\rho_q(z)$).
- **In-plane 2D Correlations:** Calculation of $g(r_{xy})$ and $S(Q_{xy})$ specifically evaluated within user-defined planar slices parallel to the confining walls. This includes species-resolved and charge-charge 2D correlations.

4.1.5. Bond orientational order parameters

Bond orientational order parameters provide a quantitative measure of the local structural organization of particles [16] and are widely used to identify crystalline, and liquid-like cluster phases.

- **3D case. Steinhardt order parameters** For each particle i , a set of complex coefficients is defined as

$$q_{lm}(i) = \frac{1}{N_b(i)} \sum_{j=1}^{N_b(i)} Y_{lm}(\hat{\mathbf{r}}_{ij}), \quad (8)$$

where Y_{lm} are the spherical harmonics, $\hat{\mathbf{r}}_{ij}$ is the unit vector connecting particles i and j , and $N_b(i)$ is the number of neighbors of particle i . The rotationally invariant bond order parameter is then defined as

$$Q_l(i) = \left(\frac{4\pi}{2l+1} \sum_{m=-l}^l |q_{lm}(i)|^2 \right)^{1/2}. \quad (9)$$

A global order parameter can be obtained by averaging over all particles:

$$Q_l = \frac{1}{N} \sum_{i=1}^N Q_l(i). \quad (10)$$

Typical choices are $l = 4$ and $l = 6$, which are sensitive to cubic and hexagonal symmetries, respectively.

- **Two-dimensional bond order parameter** For quasi-two-dimensional systems or planar structures, it is convenient to define a bond order parameter based on angular correlations[17]:

$$\phi_m(i) = \frac{1}{N_b(i)} \sum_{j=1}^{N_b(i)} e^{im\theta_{ij}}, \quad (11)$$

where θ_{ij} is the angle between the bond $\mathbf{r}_{ij}$ and a reference axis, and m is an integer corresponding to the symmetry of interest (e.g., $m = 6$ for hexagonal order). The magnitude of the order parameter is

$$|\phi_m(i)| = \left| \frac{1}{N_b(i)} \sum_{j=1}^{N_b(i)} e^{im\theta_{ij}} \right|. \quad (12)$$

A global measure is obtained by averaging over all particles:

$$\phi_m = \frac{1}{N} \sum_{i=1}^N |\phi_m(i)|. \quad (13)$$

- **Practical considerations and implementation** The definition of neighbors $N_b(i)$ is crucial and is typically based on a cutoff distance, r_{cl} (which here is defined as the first minimum of $g(r)$). Note that we will use this same separation as connectivity distance when performing the cluster analysis. Now, the parallelization will also be straightforward, each GPU thread being in charge of the calculation of the corresponding order parameters for each particle taking into account only the corresponding particle neighbors, and then averaging over configurations and finally over all particles. These last two operations are carried out on the CPU. The algorithms implemented to evaluate both the spherical harmonics in (8) or the simpler $\sin m\phi, \cos m\phi$ functions in (12) resort to stable and numerically accurate recursion formulae[18]. Results are stored in `order.dat` and `order_per_mol.dat`.

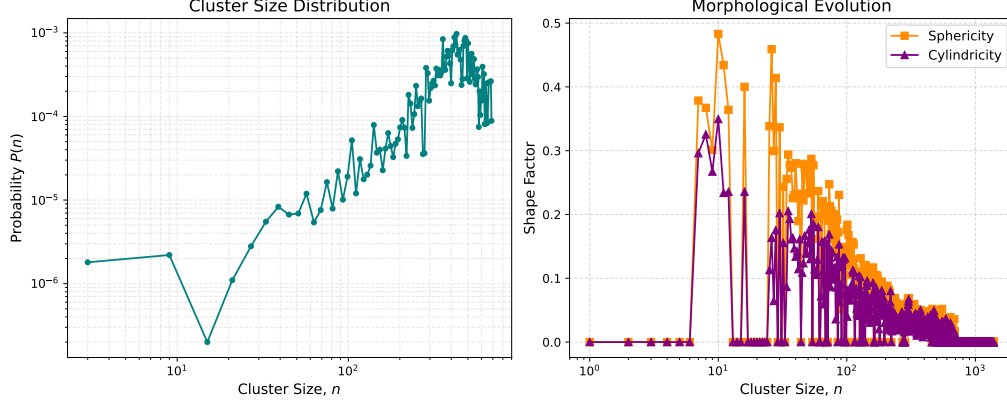


Figure 2: Evolution of cluster morphology for the 3D SALR system. Extracted from `fshape.dat` and `clustdistr.dat`, demonstrating the transition from globular to elongated structures as cluster size increases.

4.2. Cluster Analysis

This is one of the most time-consuming tasks when performing a configurational analysis in a purely serial code. When using neighbor lists the computational complexity is $O(N)$, and algorithms such as the density-based SCAN (DBSCAN) are particularly suited to a GPU implementation. Here we have followed the algorithm G-DBSCAN as developed by Andrade and coworkers[19]. In what follows we describe the key elements of our implementation.

4.2.1. Neighbor list construction

Here we use a linked cell approach[20] in order to reduce the computational complexity of the neighbor search algorithm to $O(N)$. The linked cell list is computed in the CPU and transferred to the GPU devices. This could be further optimized by transferring the full computational load of the cell list to the GPU[21], but the gain is negligible compared to other tasks, such as the Breadth-First-Search part[19] of the cluster construction. A single GPU thread is allocated to each particle and searches for its neighbors, marking all those with more than `minPts - 1` as *core* nodes, where `minPts` is the minimum number of points (particles) to consider a cluster. Neighbors are stored in a one-dimensional adjacency list which is properly and efficiently organized using Thrust library parallel prefix sum (exclusive scan) operations[22].

4.2.2. Cluster expansion

Once the neighbor list is defined and nodes are identified as core or border (connected but with less than `minPts - 1` neighbors), clusters are expanded in a parallel Breadth-First-Search fashion[19]. Each GPU thread starts from a core point, and then each level processes border points, and adds unvisited neighbors; the process is continued until no more new points are left. All points connected to a core point are considered part of the same cluster. Disconnected groups with less than `minPts` points are considered noise. The outer loop over clusters remains sequential. The algorithm scheme, taken from Ref. [23] is schematically shown in Algorithm 1.

Once clusters have been identified and classified in the corresponding data structures the program determines the cluster size distribution (basic minimum information provided), and if the number of clusters is larger than `clthresh` (defaults to 10) performs the following analysis:

Algorithm 1 Cluster expansion (parallel BFS level)

```
1: procedure EXPANDCLUSTERS(labels, core_flag, adj_list, adj_offsets)
2:   labels[·]  $\leftarrow$  -1, cluster_id  $\leftarrow$  0
3:   for each unvisited core point  $p$  do
4:     labels[ $p$ ]  $\leftarrow$  cluster_id
5:     frontier  $\leftarrow$  { $p$ }
6:     while frontier  $\neq \emptyset$  do
7:       new_frontier  $\leftarrow \emptyset$  ▷ parallel loop
8:       for each  $q$  in frontier in parallel do
9:         start  $\leftarrow$  adj_offsets[ $q$ ], end  $\leftarrow$  adj_offsets[ $q+1$ ]
10:        for  $r$  in adj_list[start:end] do
11:          if labels[ $r$ ] = -1 then
12:            labels[ $r$ ]  $\leftarrow$  cluster_id
13:            if core_flag[ $r$ ] then
14:              new_frontier  $\leftarrow$  new_frontier  $\cup$  { $r$ }
15:            end if
16:          end if
17:        end for
18:      end for
19:      frontier  $\leftarrow$  new_frontier
20:    end while
21:    cluster_id  $\leftarrow$  cluster_id + 1
22:  end for
23: end procedure
```

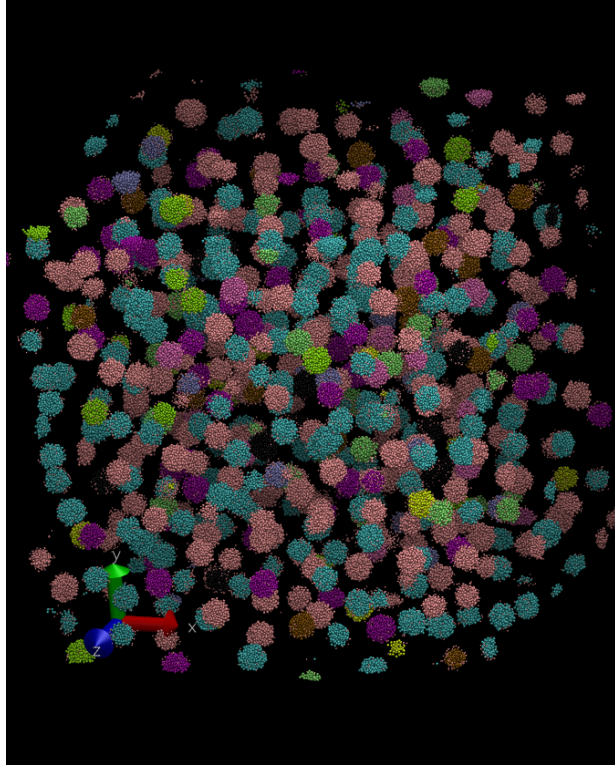


Figure 3: Simulation snapshot of the 3D SALR system rendered using the generated `last_clconf.lammpstrj`. Particles are colored by their `trj_analysis` assigned cluster IDs.

- Determine the distribution of gyration radii, defined as

$$R_g = \left\langle \sqrt{\frac{\sum_i m_i (\mathbf{r}_i - \mathbf{R}_{com})^2}{\sum_i m_i}} \right\rangle \quad (14)$$

where m_i and $\mathbf{r}_i$ are particle's mass and positions, and $\mathbf{R}_{com}$ is the clusters center of mass. Printout in `radii.dat`.

- Cluster density profile, as calculated from the center of mass, stored in `rho_prof.dat`, and the distribution of cluster internal densities, `dens.dat`.
- If order parameters are calculated, in `ord_prof_clust.dat` the corresponding spatial profiles of the order parameters, and their cumulative counterparts defined by

$$Q_l^c(r) = \frac{3}{2r^3} \int_0^r r'^2 Q_l(r') dr'. \quad (15)$$

are stored.

- Cluster-cluster correlations (g_{cl-cl}), inner cluster correlations (g_{cl}) and structure factors $S_{cl-cl}(Q)$ (in `sqcl.dat`) are also calculated if requested.

- Form factors quantifying the deviation from the spherical (or circular) shape and cylindrical shape are also determined and its distribution according to cluster size is stored in `fshape.dat`. These quantities are computed from the values of the principal axes of inertia, after diagonalization of the inertia tensor (setting all masses to one, since we only care about the shape), and are defined by

$$f_{cyl} = \sqrt{\frac{|I_{yy} - I_{xx}|}{I_{xx}^2 + I_{yy}^2 + I_{zz}^2}} \quad (16)$$

$$f_{sph} = \sqrt{\frac{(I_{xx} - \text{Tr}(\mathbf{I}))^2 + (I_{yy} - \text{Tr}(\mathbf{I}))^2}{I_{xx}^2 + I_{yy}^2 + I_{zz}^2}} \quad (17)$$

where $\mathbf{I}$ is the inertia matrix, I_{zz} is its smallest eigenvalue and Tr is the trace operator. If $f_{cyl} \approx f_{sph} \rightarrow 0$ we have a spherical (circular) shape. If only f_{cyl} is significantly close to zero, we have a cylindrical shape.

- Cluster center-of-mass trajectory output collected in `centers.lampstrj` using LAMMPS text dump format.
- A LAMMPS dump file for the last configuration in which particle IDs are correlated to cluster sizes is also produced (`last_clconf.lampstrj`) in addition of a direct dump of the last particle configuration as generated by LAMMPS (`last_conf.lampstr`) for visualization purposes.

4.2.3. Geometric identification of cluster surfaces

The final cluster configuration can additionally be filtered using the local asymmetry of the neighbor directions,

$$\alpha_i = \frac{1}{N_b(i)} \left| \sum_{j \in N(i)} \frac{\mathbf{r}_{ij}}{|\mathbf{r}_{ij}|} \right|. \quad (18)$$

The GPU evaluates this descriptor using the cluster adjacency list. Atoms assigned to a cluster and satisfying $\alpha_i \geq \text{asym_threshold}$ (default 0.5) are written to `last_brdconf.lampstrj`. Balanced neighbor directions give small values, whereas asymmetric environments can identify surface atoms and chain endpoints. This geometric criterion is distinct from the core/border classification used during DBSCAN expansion and is sensitive to the chosen connectivity cutoff and threshold.

4.3. Dynamical correlation functions

Time-dependent observables are evaluated using multiple time origins in order to improve statistical accuracy. Given a trajectory sampled at discrete times

$$t_n = n\Delta t, \quad n = 0, \dots, N_{\text{conf}} - 1,$$

correlation functions are computed as averages over particles and time origins.

4.3.1. Position self-correlation function

The position self-correlation function is commonly expressed through the mean-square displacement (MSD):

$$\text{MSD}(t) = \left\langle \frac{1}{N} \sum_{i=1}^N |\mathbf{r}_i(t_0 + t) - \mathbf{r}_i(t_0)|^2 \right\rangle_{t_0}.$$

In discrete-time form, this becomes

$$\text{MSD}(m\Delta t) = \frac{1}{N(N_{\text{conf}} - m)} \sum_{i=1}^N \sum_{n=0}^{N_{\text{conf}}-m-1} |\mathbf{r}_i(t_{n+m}) - \mathbf{r}_i(t_n)|^2.$$

This expression gives the all-origin estimator. In the implementation, the average is taken over the origins sampled by the configurable buffering scheme.

4.3.2. Velocity autocorrelation function

The velocity autocorrelation function (VACF) is defined as

$$C_v(t) = \left\langle \frac{1}{N} \sum_{i=1}^N \mathbf{v}_i(t_0 + t) \cdot \mathbf{v}_i(t_0) \right\rangle_{t_0}.$$

Its discrete estimator reads

$$C_v(m\Delta t) = \frac{1}{N(N_{\text{conf}} - m)} \sum_{i=1}^N \sum_{n=0}^{N_{\text{conf}}-m-1} \mathbf{v}_i(t_{n+m}) \cdot \mathbf{v}_i(t_n).$$

The VACF provides access to transport coefficients such as the diffusion constant via the Green–Kubo relation, and the corresponding frequency spectrum, $Z(\omega)$, using a time resolved Fourier transform, to which purpose we use here the FFTW library[24].

4.3.3. Shear Viscosity from Green-Kubo Relations

The shear viscosity η , can be determined through the Green-Kubo formulation, which relates the transport coefficient to the time-integral of the auto-correlation function of its corresponding flux at equilibrium. In the limit of linear response theory, the shear viscosity η is given by the integral of the Stress Auto-Correlation Function (SACF):

$$\eta = \frac{V}{k_B T} \int_0^\infty \langle P_{\alpha\beta}(t) P_{\alpha\beta}(0) \rangle dt \quad (19)$$

where k_B is the Boltzmann constant and T is the absolute temperature. For isotropic fluids, statistical accuracy is improved by averaging over the three independent off-diagonal components: P_{xy} , P_{xz} , and P_{yz} . These quantities can be evaluated from the forces, but here we have chosen to simply read in the corresponding compute (see 4.4 below) calculated by LAMMPS and stored in the NetCDF dump file as a *per atom* quantity.

As done for other dynamic correlations, in order to maximize statistical sampling from a finite trajectory of N_T total steps, we employ the method of multiple time origins. For a correlation lag time $\tau = m\Delta t$, the discretized SACF, $C(m\Delta t)$, is calculated as:

$$C(m\Delta t) = \frac{1}{3|O_m|} \sum_{n \in O_m} \sum_{\alpha\beta \in \{xy, xz, yz\}} P_{\alpha\beta}(t_{n+m}) P_{\alpha\beta}(t_n), \quad (20)$$

where O_m denotes the sampled time origins. The pressure components are system-wide quantities obtained by summing the per-atom stress contributions and applying the volume and unit conversion; there is no additional average over atoms in this collective SACF. The GPU distributes the correlation work over buffered origins.

The shear viscosity is then computed using a numerical integration scheme, here the trapezoidal rule, up to a truncation time $M\Delta t$ where the correlation has sufficiently decayed:

$$\eta = \frac{V\Delta t}{k_B T} \left[\frac{1}{2} C(0) + \sum_{m=1}^{M-1} C(m\Delta t) + \frac{1}{2} C(M\Delta t) \right] \quad (21)$$

In practice, the choice of the maximum lag time $M\Delta t$ is critical; it must be large enough to capture the decay of the SACF but small enough to avoid the accumulation of statistical noise that occurs at long times.

4.3.4. Self-intermediate scattering function

The self part of the intermediate scattering function is defined as

$$F_s(Q, t) = \left\langle \frac{1}{N} \sum_{i=1}^N e^{i\mathbf{Q} \cdot [\mathbf{r}_i(t_0+t) - \mathbf{r}_i(t_0)]} \right\rangle_{t_0}.$$

Its discrete estimator becomes

$$F_s(Q, m\Delta t) = \frac{1}{N(N_{\text{conf}} - m)} \sum_{i=1}^N \sum_{n=0}^{N_{\text{conf}}-m-1} e^{i\mathbf{Q} \cdot [\mathbf{r}_i(t_{n+m}) - \mathbf{r}_i(t_n)]}.$$

For isotropic systems, angular averaging over wavevectors of equal magnitude $Q = |\mathbf{Q}|$ is performed to improve statistics.

4.3.5. Collective intermediate scattering function

The collective intermediate scattering function is obtained from density fluctuations:

$$F(Q, t) = \frac{1}{N} \langle \rho(\mathbf{Q}, t_0 + t) \rho(-\mathbf{Q}, t_0) \rangle_{t_0},$$

where

$$\rho(\mathbf{Q}, t) = \sum_{j=1}^N e^{i\mathbf{Q} \cdot \mathbf{r}_j(t)}.$$

The corresponding discrete estimator reads

$$F(Q, m\Delta t) = \frac{1}{N(N_{\text{conf}} - m)} \sum_{n=0}^{N_{\text{conf}}-m-1} \rho(\mathbf{Q}, t_{n+m}) \rho(-\mathbf{Q}, t_n).$$

4.3.6. Partial intermediate scattering functions

For multicomponent systems, partial density functions are defined as

$$\rho_\alpha(\mathbf{Q}, t) = \sum_{j \in \alpha} e^{i\mathbf{Q} \cdot \mathbf{r}_j(t)},$$

leading to

$$F_{\alpha\beta}(Q, t) = \frac{1}{\sqrt{N_\alpha N_\beta}} \left\langle \rho_\alpha(\mathbf{Q}, t_0 + t) \rho_\beta(-\mathbf{Q}, t_0) \right\rangle_{t_0}.$$

These functions define species cross-correlations. The present dynamic output contains the total coherent and self functions; the coherent kernel combines species amplitudes using the scattering lengths bsc. The expressions above for total density assume unit scattering lengths.

Time correlations are accumulated directly using reusable buffers with configurable origin spacing and lag windows. This limits storage but does not, by itself, imply an $O(N_{\text{conf}} \log N_{\text{conf}})$ correlation algorithm. FFTW[24] is used subsequently to transform the accumulated functions into frequency spectra. Host accumulation and FFT processing use double precision. Dynamic scattering and the associated current calculations are disabled for confined systems.

4.3.7. Longitudinal and transverse current correlations

For a nonzero wavevector, define $\hat{\mathbf{Q}} = \mathbf{Q}/Q$, $v_{L,i} = \mathbf{v}_i \cdot \hat{\mathbf{Q}}$ and $\mathbf{v}_{T,i} = \mathbf{v}_i - v_{L,i} \hat{\mathbf{Q}}$. For unit scattering lengths, the microscopic currents are

$$j_L(\mathbf{Q}, t) = \sum_i v_{L,i}(t) e^{i\mathbf{Q} \cdot \mathbf{r}_i(t)}, \quad \mathbf{j}_T(\mathbf{Q}, t) = \sum_i \mathbf{v}_{T,i}(t) e^{i\mathbf{Q} \cdot \mathbf{r}_i(t)}. \quad (22)$$

Their collective correlations in d spatial dimensions are

$$J_L(Q, t) = \frac{1}{N} \text{Re} \langle j_L(\mathbf{Q}, t_0 + t) j_L(\mathbf{Q}, t_0)^* \rangle, \quad (23)$$

$$J_T(Q, t) = \frac{1}{N(d-1)} \text{Re} \langle \mathbf{j}_T(\mathbf{Q}, t_0 + t) \cdot \mathbf{j}_T(\mathbf{Q}, t_0)^* \rangle. \quad (24)$$

The self parts retain only equal-particle terms in these products. The factor $1/(d-1)$ averages the transverse polarizations: it is $1/2$ in 3D and unity in 2D. For an equilibrium single-component system of particle mass m , the self zero-time limits are $J_{L,s}(Q, 0) = J_{T,s}(Q, 0) = k_B T/m$.

The CUDA kernel evaluates these quantities alongside the density correlations using the same wavevector shells and buffered origins. For mixtures, the collective currents use bsc weighting, whereas the self currents are averaged over particles without that weighting. Current calculations require velocities in the trajectory and the SQW module. Their presence is detected by the reader; no additional current-activation namelist is required. Time-domain results are written to `jqt.dat` and `jtqt.dat`; their Fourier spectra $C_L(Q, \omega)$ and $C_T(Q, \omega)$, together with the self parts, are written to `clqw.dat` and `ctqw.dat`.

4.3.8. Spectral processing and numerical consistency

Two optional INPUT_SQW switches control the density spectra. With `subtract_plateau` enabled, the code estimates the plateau from the last five available correlation samples (or all samples if fewer than five are available) and subtracts it before transformation. With `norm_sqw` enabled, it divides the transform input by the original zero-time amplitude, $F(Q, 0)$ or $F_s(Q, 0)$.

Both switches default to false and can be combined; the time-domain files retain the unmodified correlations. A finite-window tail estimate should only be interpreted as a non-ergodicity plateau if the sampled time window supports that interpretation.

The corrected FFT processing and output cover the nonnegative frequency range through the Nyquist limit $f_{\text{Nyq}} = 1/(2\Delta t)$, with $\omega = 2\pi f$ for angular frequency. A separate frequency grid is retained for the VACF spectrum. Stress-correlation buffers are initialized explicitly, and Green–Kubo viscosity uses the reduced-unit prefactor V/T for LAMMPS 1j units, with viscosity reported in $\epsilon\tau/\sigma^3$; dimensional output is reported in bar ps.

4.4. Thermodynamic Quantities (if present in trajectory)

If the LAMMPS NetCDF dump contains *per atom* quantities such as energy and/or pressure in addition to kinetic energy evaluation (when velocities are included in the dump), the code also computes:

- Potential energy statistics per atom and per cluster if cluster analysis is also performed (generating `distUcl_N.dat` and `distUcltot.dat`).
- Stress tensor and pressure evaluation.

As mentioned, this requires LAMMPS to perform the appropriate computes as follows:

```
compute ener all pe/atom
compute stress all stress/atom NULL
dump trj1 all netcdf ${Ndump} run.nc id type mol x y z vx vy vz q c_stress[*]
c_ener
```

5. Installation and Usage

5.1. Requirements

- NetCDF library installation (Fortran and C interfaces), Fortran interface compiled with NVIDIA's `nvfortran`, <https://www.unidata.ucar.edu/software/netcdf>.
- FFTW3 library and development components, and BLAS/LAPACK libraries.
- Fortran compiler with F2008 support (e.g., `nvfortran`).
- C++ compiler (C++17 or newer, e.g., `nvcc`).
- NVIDIA HPC Software Development Kit (includes `nvfortran`), <https://developer.nvidia.com/hpc-sdk>

The supplied build targets NVIDIA compute capabilities 7.5, 8.0, 8.6, 8.9, 9.0 and 12.0 in the Fortran kernels; a toolchain supporting the selected targets is required. The CUDA C++ helper includes native targets through 9.0 and forward-compatible PTX. Architecture flags can be adjusted for the installed hardware and compiler.

5.2. Compilation

Typical build commands:

```
git clone https://github.com/elomba/trj_analysis
cd trj_analysis
# configure NetCDF, FFTW3, and CUDA paths
make
```

For molecular systems, the optional `mol` dump field preserves molecule IDs in exported configurations.

This will build a GPU-enabled program `trj_analysis` in the `trj_analysis/bin` directory.

6. Program input description

Input parameters are provided through Fortran namelists. Defaults are assigned internally whenever possible; otherwise parameters must be explicitly defined when activating the corresponding module. Some parameters inherit values from trajectory metadata and therefore become available only after the input file has been parsed. Unless otherwise specified, all lengths are expressed in simulation units.

6.1. Global namelist: *INPUT*

This namelist is mandatory.

Parameter	Type	Default	Notes
<code>trj_input_file</code>	string	required	NetCDF trajectory filename
<code>log_output_file</code>	string	required	Output log file
<code>ndim</code>	integer	required	Automatically overridden if detected in trajectory
<code>nsp</code>	integer	required	Updated after species detection if available
<code>ncfs_from_to</code>	integer(3)	0,0,0	Config range: [No. confs, start, end]. Default: all
<code>rdf_sq_cl_dyn_sqw_conf_ord</code>	logical(7)	F,F,F,F,F,F,F	Module activation flags
<code>nthread</code>	integer	64	CUDA threads per block
<code>nqw</code>	integer	0	Used only if SQW module active
<code>nslice</code>	integer	1	Number of slices for confinement or profiling
<code>norder</code>	integer	1	Maximum order for Steinhardt parameters
<code>ener_name</code>	string	none	Required only if energy histograms requested
<code>press_name</code>	string	none	Required only if stress analysis requested
<code>potnbins</code>	integer	100	Energy histogram bins
<code>potengmargin</code>	real	0.0	Energy histogram margin
<code>rcl</code>	real	none	Required if clustering or order analysis active
<code>periodic</code>	logical array	T,T,T	Periodic boundary conditions per dimension
<code>topol</code>	logical	F	When true molecular topology is analyzed
<code>system_data_file</code>	string	<code>system.data</code>	Topology data file
<code>is_hs</code>	logical	F	Hard-sphere thermodynamic labels
<code>model</code>	string	empty	HS selects hard-sphere mode
<code>nprint</code>	integer	10	Printout frequency

6.2. Species namelist: *INPUT_SP*

This namelist is read unconditionally. Specify the selected LAMMPS atom type IDs and their masses, including for single-component systems.

Parameter	Type	Default	Notes
sp_types_selected	integer array	required	LAMMPS type IDs for selected species
mat	real array	required	Atomic masses for each selected species
rigid	logical	F	If true rigid molecular units are present
nmrigid	integer	0	Number of types of rigid units
rigid_mols	integer array	–	IDs of rigid unit types

6.3. Radial distribution functions: *INPUT_RDF*

Read when RDF, cluster analysis, or confinement is enabled.

Parameter	Type	Default	Notes
deltar	real	required	Radial grid spacing
rcrdf	real	required	Must satisfy $r_c < L/2$
nrandom	integer	0	Number of random origins for fluctuation analysis

6.4. Static structure factor: *INPUT_SQ*

Read when SQ, cluster analysis, or SQW is enabled.

Parameter	Type	Default	Notes
qmin	real	required	Threshold for isotropic Q-sampling
qmax	real	required	Maximum Q value
bsc	real array	1.0	Coherent scattering lengths per species

6.5. Cluster analysis: *INPUT_CL*

Activated when clustering flag is true.

Parameter	Type	Default	Notes
dcl	real	required	Grid spacing for cluster distributions
minPts	integer	$2 \times \text{ndim} + 1$	Core-neighborhood threshold
ndrclus	integer	1	Bins for cluster radial profiles
cl_thresh	integer	10	Minimum cluster count for correlations
geometry	logical	T	Enables geometry and shape computations
asym_threshold	real	0.5	Surface asymmetry cutoff

6.6. Confinement analysis: *INPUT_CONF*

Activated when confinement flag enabled.

Parameter	Type	Default	Notes
zslice	real array	none	Explicit slice positions across the z-axis
zgrid	real	none	Grid spacing for profiles

6.7. Dynamical correlations: *INPUT_DYN*

Read when DYN or SQW is enabled. Velocities and stress availability are detected from the trajectory; they are not switches in this namelist.

Parameter	Type	Default	Notes
nbuffer	integer	2	Number of time origins for correlation
tmax	real	-1.0	Max correlation window (ps or LJ time)
tmaxp	real	-1.0	Max time for viscosity windowing
tlimit	real	-1.0	Global time limit for buffer averaging
jump	integer	1	Config stride between buffers

6.8. Dynamic structure factor: *INPUT_SQW*

Activated when SQW module enabled.

Parameter	Type	Default	Notes
qw	real array	required	Q-values for $F(Q, t)$ and $S(Q, \omega)$
tmqw	real array	0.0	Q-dependent windows; all zero uses tmax
subtract_plateau	logical	F	Subtract tail mean before density FFT
norm_sqw	logical	F	Normalize density FFT input by value at $t = 0$

6.9. Bond-orientational order: *INPUT_ORDER*

Activated when order-analysis flag enabled.

Parameter	Type	Default	Notes
orderp	integer array	required	Harmonic indices ℓ for Steinhardt parameters
print_orderp	logical	F	Enables per-particle output
nnbond	integer	0	Fixed neighbor count (0 = use cutoff rcl)
rclcl	real	0.0	Cutoff for inter-cluster neighbors

6.10. Implicit and trajectory-derived defaults

Some parameters are determined automatically after reading trajectory metadata:

- number of configurations N_{conf}
- simulation box dimensions
- particle number N
- species populations N_α
- dimensionality (if stored in trajectory)
- accessible correlation time window

These quantities override user defaults when present.

7. Output files

Table 1 lists the output files generated by `trj_analysis`. Files are produced only when the corresponding analysis modules are activated.

Table 1: Output files generated by `trj_analysis`.

File name	Observable	Conditions
<i>Thermodynamics & General Output</i>		
analysis.log	Execution log and parameter summary	Always written
thermo_run.dat	Time series of instantaneous T , P , and internal energy	if P , and U in dump
pot_energy.dat	Global per-atom potential energy distributions	ener_name defined

<i>Structural Analysis</i>		
gmixsim.dat	Pair distribution functions $g_{\alpha\beta}(r)$	RDF module active
s2n.dat	Number fluctuation data	nrandom > 0
sq.dat	Total static structure factor $S(Q)$, $S_{NN}(Q)$, $S_{cc}(Q)$	SQ module active
sqmix.dat	Species-resolved partial structure factors	SQ module active
sqcl.dat	Cluster-center structure factor $S_{cl}(Q)$	CL + SQ modules
order.dat	Averaged Steinhardt order parameters $\langle Q_l \rangle$	ORDER module active
order_per_mol.dat	Per-particle order parameters	print_orderp = T
<i>Directional & Confinement Analysis</i>		
densprof.dat	Density profile along the confinement direction (z)	CONF module active
qdens.dat	Total and species-resolved charge density profiles	CONF + charges present
gxy_i-j.dat	2D RDFs calculated within slices in the xy plane	CONF + RDF
gxy_qq.dat	2D charge-charge RDF within slices in the xy plane	CONF + RDF + charges
sqxy.dat	2D structure factor calculated within slices in the xy plane	CONF + SQ
sqxy_qq.dat	2D charge-charge structure factor within slices in xy plane	CONF + SQ + charges
sqpxy_i-i.dat	Partial 2D structure factors per species in the xy plane	CONF + SQ
<i>Cluster Analysis</i>		
clustdistr.dat	Cluster size distribution	CL module active
radii.dat	Distribution of cluster radii of gyration	CL module active
rho prof.dat	Average cluster radial density profiles	CL module active
fshape.dat	Cluster shape descriptors (sphericity, cylindricity)	geometry = T
distUcl_N.dat	Distribution of internal energy per particle	CL + ener_name
distUcltot.dat	Distribution of total cluster internal energy	CL + ener_name
clusevol.dat	Time evolution of cluster counts and clustered fractions	CL module active
order_per_clust.dat	Order parameters averaged per cluster	CL + ORDER
ordprof_clust.dat	Steinhardt parameter profiles across cluster slices	CL + ORDER
ordprof_clcum.dat	Cumulative Steinhardt profiles	CL + ORDER
centers.lampstrj	Cluster center-of-mass trajectory for visualization	CL module active
last_brdconf.lampstrj	Final cluster surface atoms	CL module active
last_clconf.lampstrj	Final configuration with particles colored by cluster ID	CL module active
<i>Dynamical Correlations</i>		
dyn.dat	Mean squared displacement and velocity autocorrelation	DYN module active
dynw.dat	VACF spectrum $Z(\omega)$	DYN + velocities
fqt.dat	Coherent intermediate scattering function $F(Q, t)$	SQW module active
fskt.dat	Self-intermediate scattering function $F_s(Q, t)$	SQW module active
sqw.dat	Dynamic structure factors $S(Q, \omega)$ and $S_s(Q, \omega)$	SQW module active
jqt.dat, jtqt.dat	Longitudinal/transverse currents (total and self)	SQW + velocities
clqw.dat, ctqw.dat	Longitudinal/transverse current spectra	SQW + velocities
viscor.dat	Stress autocorrelation and shear viscosity $\eta(t)$	DYN + press_name

8. Example Usage and validation

An example run on a NetCDF trajectory file:

```
./bin/trj_analysis input.nml GPU_id
```

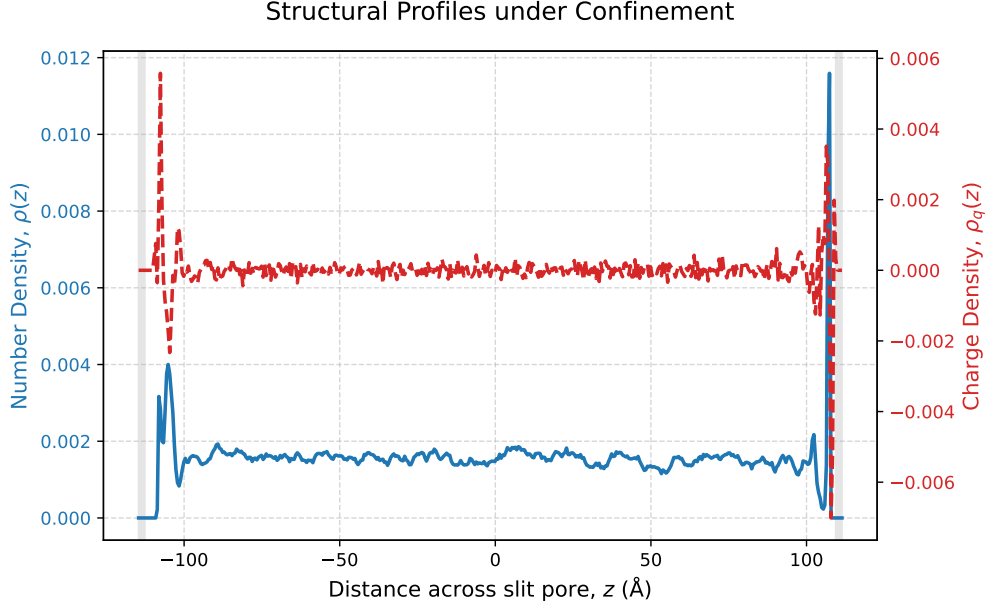


Figure 4: Density $\rho(z)$ and charge density $\rho_q(z)$ profiles for an ionic liquid confined between charged plates, extracted from `densprof.dat` and `qdens.dat` using the CONF module.

where `GPU_id` is the number identifying a GPU device (as given by `nvidia-smi`), and `input.nml` is a file containing the namelists specified above.

For the original submission, the correctness of observables was verified using a variety of canonical MD test systems. For structural observables like pair distribution functions and static structure factors, comparisons to reference calculations showed agreement within statistical uncertainties. Dynamical observables such as time correlation functions and MSDs similarly matched expected behavior.

A comprehensive set of validation examples is included in the repository under the directory `examples/` (cf tree of Figure 6). The examples cover several types of systems commonly encountered in soft-matter and molecular simulations. Also to showcase the capabilities of the toolkit, we have extracted data from the provided validation examples and generated representative figures for some of the examples cited below.

- SALR systems (short-range attraction / long-range repulsion) exhibiting cluster formation, both in 3D and 2D. These examples are taken from Refs. [25] and [26] respectively. In Figure 2 we observe the size distribution of a system of 3D SALR clusters (left) and the distribution of shape factors (right). The distribution indicates a dominance of large spherical clusters, with important deviations for small sizes. A snapshot that illustrates the clusters is presented in Figure 3.
- A simple binary Lennard-Jones mixture in which LAMMPS 1j-units are used.
- A coarse grained model of ionic liquid and TIP4p/2005 water between two plates at con-

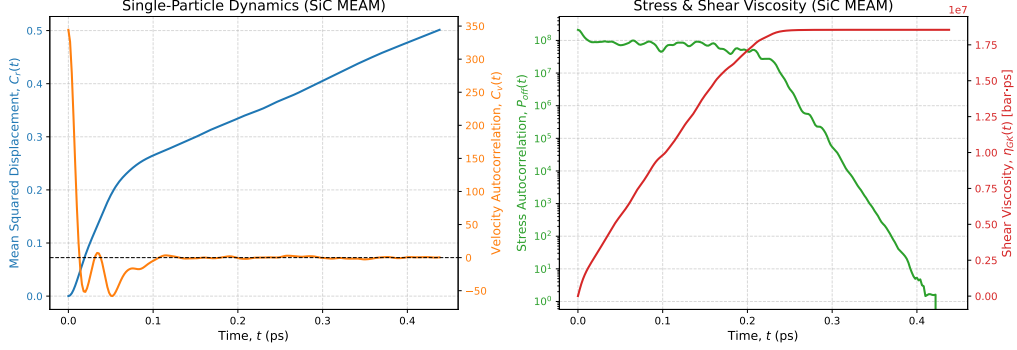


Figure 5: Dynamical properties and transport coefficients for a silicon carbide (SiC) alloy modeled with a MEAM potential. (Left) Single-particle dynamics showing the mean squared displacement (MSD) and the highly oscillatory velocity autocorrelation function (VACF) typical of microscopic vibrational dynamics, evaluated using the multi-origin buffering algorithm and extracted from `dyn.dat`. (Right) Collective dynamics illustrating the rapid decay of the off-diagonal stress autocorrelation function (SACF) alongside the convergence of the running integral for the Green-Kubo shear viscosity (η_{GK}), extracted from `viscor.dat`.

stant potential taken from [27]. The corresponding number and charge density profiles is plotted in Figure 4. The marked charge asymmetry induced by the potential difference is clearly visible.

- The dynamics of a molten SiC alloy described using a MEAM potential [28] was also analyzed and in Figure 5 we can see the behavior of the mean square displacement and velocity autocorrelation (left), and stress autocorrelation and shear viscosity (right). The curves display a good convergence that results from the use of 40 time origins, separated by 20 stored configurations (2 ps). `metal` units are used.

Table 2: Benchmark comparison between CPU and GPU implementations for structural analysis calculations.

	97,336 particles			216,000 particles		
Method	CPU (s)	GPU (s)	Speedup	CPU (s)	GPU (s)	Speedup
$g(r)$	8.40	0.03	280×	17.40	0.165	105×
$Q_4, Q_6, Q_8, Q_{10}, Q_{12}$	0.02	0.003	6.7×	0.17	0.008	21×
$S(Q)$	4.45	0.09	49×	22.00	0.29	76×
Cluster analysis	26.6	0.12	220×	64.00	0.25	250×

The revision adds ideal BCC, FCC and simple-cubic bond-order examples in `examples/order`, accompanying the corrected harmonic recursion. The script `tools/check_clqw_sqw.py` provides checks and plots for density and current spectra, and `doc/report_current_correlations.tex` documents their conventions and equilibrium limits. These resources supplement the original examples; the figures and numerical benchmarks reproduced here are from the submission and do not constitute new validation runs for version 1.7.6.

9. Performance

The submission-version performance was benchmarked using an Intel Xeon Gold 6548Y+ dual processor and an NVIDIA RTX Pro 4500 Blackwell with Compute Capability 12.0. In Table 2 we present a comparison for various structural properties and cluster analysis. For the calculation of the pair distribution function and orientational bond order parameters[16] LAMMPS built-in `compute rdf` and `compute orientorder/bond` were employed by means of a rerun over a stored trajectory. This allows the calculation of $g(r)$ values beyond the potential cutoff and does not include the computational cost of the simulation itself. Moreover, I/O time was also subtracted from the total CPU time. The corresponding LAMMPS scripts are collected in the `tools/` directory. The static structure factor was computed using the script `fast_sq_freud_netcdf.py` included in the directory `tools/`. This script uses a optimized routine for direct structure factor evaluation from Freud 3.5.0 package[6], `freud.diffraction.StaticStructureFactorDirect`. From the table it is evident that GPU acceleration yields a very significant reductions in runtime for histogram and correlation tasks compared to CPU-only routines, allowing analysis of large trajectories in practical wall-clock times. Speed-up for dynamic correlations are not included, since in this version parallelization is only implemented at the level of multiple time-origin selection, and results are comparable to optimized approaches such as the Fast Correlation Analysis using Fast Fourier Transforms[29]. A GPU version of this approach will be incorporated in a future release. Finally, the CPU analysis of clusters was carried out implementing a DBSCAN algorithm in a highly optimized code that uses NUMBA libraries [30, 23].

10. Availability and Documentation

The code for `trj_analysis` is available at:

https://github.com/elomba/trj_analysis

The repository also records release changes in `Changelog.md`. It includes example input files, output examples, documentation, and validation cases to facilitate usage and extension, all organized as indicated in the diagram of Figure 6. Both the parent directory, `tools/` and `examples/` directories include the corresponding `README.md` files explaining the contents and basic instructions.

11. Conclusions

`trj_analysis` provides a scalable, GPU-accelerated toolkit for analyzing LAMMPS NetCDF trajectories. The combination of Fortran, C++, and GPU support enables efficient processing for large systems and long trajectories yielding a large panoply of physical quantities. The modular structure supports extensibility for new observables and workflows. The current release extends the dynamical analysis to longitudinal and transverse currents and improves spectral processing and numerical consistency. Geometric surface detection complements the existing cluster descriptors. Further work includes FFT-based correlation accumulation and more detailed topological analysis of complex clusters.

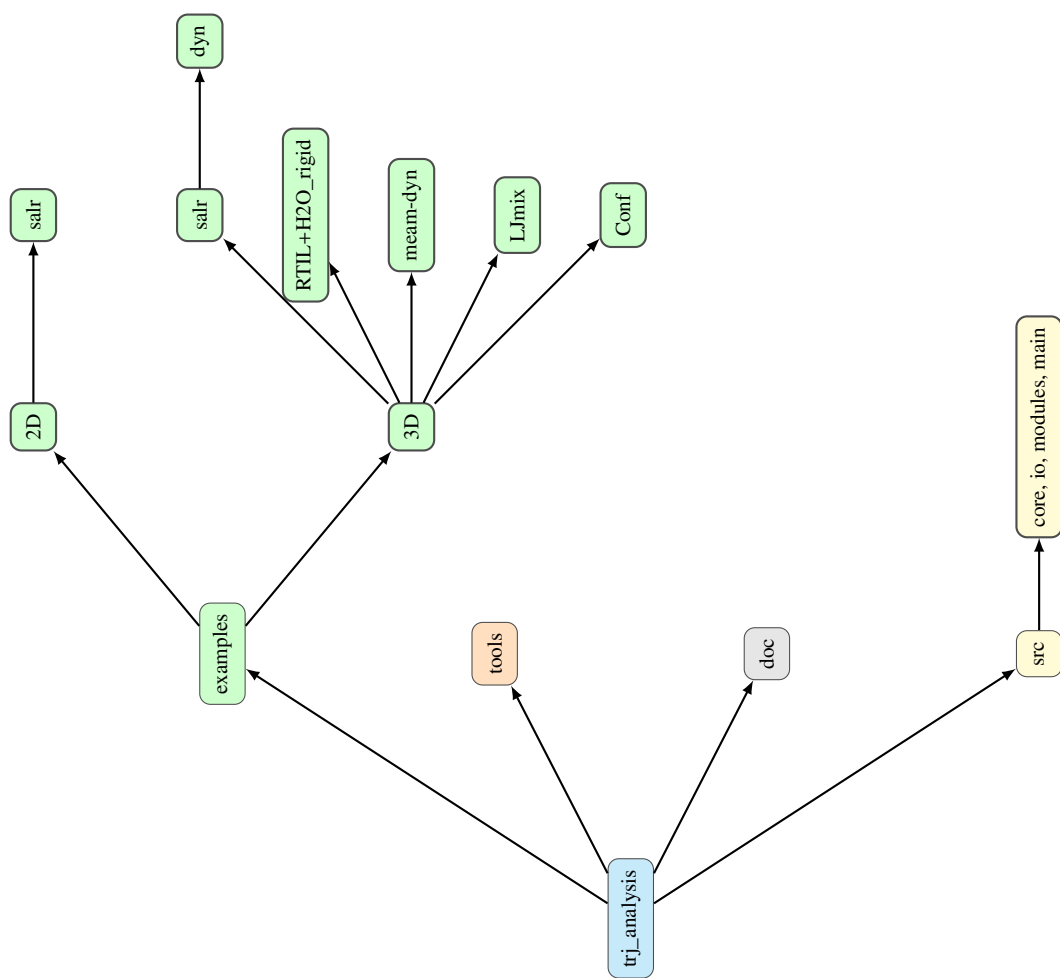


Figure 6: Directory structure of the `trj_analysis` repository. The software is organized into modules for structural, clustering, order-parameter and dynamical analysis. Example workflows and trajectory utilities are provided in the `examples/` and `tools/` directories.

12. Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work the authors used Gemini 3.1 in order to generate input parameter and output file tables and flow/directory structure diagrams. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article. During revision, the authors also used OpenAI Codex to compare the repository changes with the manuscript and assist with updating the text, equations and parameter tables.

Acknowledgements

EL and ADP acknowledge the financial support from Grant No. PID2023-151751NB-I00 funded by MICIU/AEI/10.13039/501100011033 and by “ERDF A way of making Europe.” We would also like to acknowledge the Galicia Supercomputing Center (CESGA) for the generous access to their computer facilities.

References

- [1] R. Rew, G. Davis, Netcdf: an interface for scientific data access, *IEEE Computer Graphics and Applications* 10 (4) (1990) 76–82. doi:[10.1109/38.56302](https://doi.org/10.1109/38.56302).
- [2] A. P. Thompson, H. M. Aktulga, R. Berger, D. S. Bolintineanu, W. M. Brown, P. S. Crozier, P. J. in 't Veld, A. Kohlmeyer, S. G. Moore, T. D. Nguyen, R. Shan, M. Stevens, J. Tranchida, C. Trott, S. J. Plimpton, LAMMPS - a flexible simulation tool for particle-based materials modeling at the atomic, meso, and continuum scales, *Comput. Phys. Commun.* 271 (2021) 108171. doi:[10.1016/j.cpc.2021.108171](https://doi.org/10.1016/j.cpc.2021.108171).
- [3] N. Michaud-Agrawal, E. J. Denning, T. B. Woolf, O. Beckstein, Mdanalysis: A toolkit for the analysis of molecular dynamics simulations, *Journal of Computational Chemistry* 32 (10) (2011) 2319–2327. doi:[10.1002/jcc.21787](https://doi.org/10.1002/jcc.21787).
- [4] R. J. Gowers, M. Linke, J. Barnoud, T. E. Reddy, M. N. Melo, S. L. Seyler, J. Domański, D. L. Dotson, S. Buchoux, I. M. Kenney, O. Beckstein, Mdanalysis: A python package for the rapid analysis of molecular dynamics simulations, *Proceedings of the 15th Python in Science Conference* (2016) 98–105doi:[10.25080/Majora-629e541a-00e](https://doi.org/10.25080/Majora-629e541a-00e).
- [5] R. T. McGibbon, K. A. Beauchamp, M. P. Harrigan, C. Klein, J. M. Swails, C. X. Hernández, C. R. Schwantes, L.-P. Wang, T. J. Lane, V. S. Pande, Mdtraj: A modern open library for the analysis of molecular dynamics trajectories, *Biophysical Journal* 109 (8) (2015) 1528–1532. doi:[10.1016/j.bpj.2015.08.015](https://doi.org/10.1016/j.bpj.2015.08.015).
- [6] V. Ramasubramani, B. D. Dice, E. S. Harper, M. P. Spellings, J. A. Anderson, S. C. Glotzer, freud: A software suite for high throughput analysis of particle simulation data, *Computer Physics Communications* 254 (2020) 107275. doi:<https://doi.org/10.1016/j.cpc.2020.107275>.

- [7] J. A. Anderson, J. Glaser, S. C. Glotzer, HOOMD-blue: A Python package for high-performance molecular dynamics and hard particle Monte Carlo simulations, *Computational Materials Science* 173 (2020) 109363. doi:[10.1016/j.commatsci.2019.109363](https://doi.org/10.1016/j.commatsci.2019.109363).
- [8] T. D. Romo, A. Grossfield, Loos: An extensible platform for the structural analysis of simulations, *Journal of Computational Chemistry* 30 (8) (2009) 1311–1321. doi:[10.1002/jcc.21287](https://doi.org/10.1002/jcc.21287).
- [9] S. O. Yesylevskyy, Pteros 2.0: Evolution of the fast parallel molecular analysis library for c++ and python, *Journal of Computational Chemistry* 40 (13) (2019) 1632–1643. doi:[10.1002/jcc.25857](https://doi.org/10.1002/jcc.25857).
- [10] A. Stukowski, M. Hütter, et al., Ovito: Open visualization tool, *Computer Physics CommunicationsSoftware* available at <https://www.ovito.org> (2025).
- [11] K. Reuter, J. Köfinger, CADISHI: Fast parallel calculation of particle-pair distance histograms on CPUs and GPUs, *Computer Physics Communications* 236 (2019) 274–284. doi:[10.1016/j.cpc.2018.10.018](https://doi.org/10.1016/j.cpc.2018.10.018).
- [12] S. Torquato, Hyperuniform states of matter, *Phys. Rep.* 745 (2018) 1–95. doi:[10.1016/j.physrep.2018.03.001](https://doi.org/10.1016/j.physrep.2018.03.001).
- [13] J. L. Lebowitz, J. K. Percus, Ensemble dependence of fluctuations with application to machine computations, *Physical Review* 122 (1961) 1675–1691. doi:[10.1103/PhysRev.122.1675](https://doi.org/10.1103/PhysRev.122.1675).
- [14] L. Wang, Y. Deng, N. Prokof'ev, B. Svistunov, Finite-size effects in canonical and grand-canonical ensembles, *Physical Review E* 96 (2017) 042131. doi:[10.1103/PhysRevE.96.042131](https://doi.org/10.1103/PhysRevE.96.042131).
- [15] D. M. Rogers, Extension of kirkwood-buff theory to the canonical ensemble, *J. Chem. Phys.* 148 (5) (Feb. 2018). doi:[10.1063/1.5011696](https://doi.org/10.1063/1.5011696).
- [16] P. J. Steinhard, D. R. Nelson, M. Ronchetti, Bond-orientational order in liquids and glasses, *Phys. Rev. B* 28 (1983) 784 – 805. doi:[10.1103/PhysRevB.28.784](https://doi.org/10.1103/PhysRevB.28.784).
- [17] D. R. Nelson, B. I. Halperin, Dislocation-mediated melting in two dimensions, *Physical Review B* 19 (5) (1979) 2457–2484. doi:[10.1103/physrevb.19.2457](https://doi.org/10.1103/physrevb.19.2457).
- [18] S. Winczewski, J. Dziedzic, J. Rybicki, A highly-efficient technique for evaluating bond-orientational order parameters, *Comput. Phys. Commun.* 198 (2016) 128–138. doi:[10.1016/j.cpc.2015.09.009](https://doi.org/10.1016/j.cpc.2015.09.009).
- [19] G. Andrade, G. Ramos, D. Madeira, R. Sachetto, R. Ferreira, L. Rocha, G-DBSCAN: A GPU accelerated algorithm for density-based clustering, *Procedia Computer Science* 18 (2013) 369–378. doi:[10.1016/j.procs.2013.05.200](https://doi.org/10.1016/j.procs.2013.05.200).
- [20] M. Allen, D. Tildesley, *Computer Simulation of Liquids*, Clarendon, Oxford, 1987.

- [21] L. Schwiebert, E. Hailat, K. Rushaidat, J. Mick, J. Potoff, An efficient cell list implementation for monte carlo simulation on gpus, arXiv:1408.3764 (2014). doi:<http://arxiv.org/abs/1408.3764>.
- [22] NVIDIA Corporation, Thrust: Parallel algorithms library, <https://docs.nvidia.com/cuda/thrust/>, accessed: 2026-03-23 (2024).
- [23] R. Lomba-Moreno, *Diseño e implementación del algoritmo gpu-dbscan para la identificación y clasificación de agregados en imágenes*, Bsc thesis, E.T.S. de Ingenieros Informáticos (UPM) (2026).
URL <https://oa.upm.es/94883/>
- [24] M. Frigo, S. G. Johnson, The design and implementation of fftw3, Computer Physics Communications 178 (10) (2008) 685–699. doi:[10.1016/j.cpc.2007.10.016](https://doi.org/10.1016/j.cpc.2007.10.016).
- [25] A. Díaz-Pozuelo, D. González-Salgado, E. Lomba, On the build-up of effective hyperuniformity from large globular colloidal aggregates, J. Chem. Phys. 162 (7) (2025) 074903. doi:[10.1063/5.0249688](https://doi.org/10.1063/5.0249688).
- [26] C. Bores, A. Diaz-Pozuelo, E. Lomba, Association and phase transitions in simple models for biological and soft matter condensates, J. Chem. Phys. 164 (5) (Feb. 2026). doi:[10.1063/5.0310298](https://doi.org/10.1063/5.0310298).
- [27] E. Lomba, I. Cardeñosa, L. M. Varela, Solvation vs concentration fluctuations: confined dilute electrolytes near demixing at constant potential, Mol. Phys. 122 (2024) e2376895. doi:[10.1080/00268976.2024.2376895](https://doi.org/10.1080/00268976.2024.2376895).
- [28] M. I. Baskes, Modified embedded-atom potentials for cubic materials and impurities, Physical Review B 46 (5) (1992) 2727–2742. doi:[10.1103/PhysRevB.46.2727](https://doi.org/10.1103/PhysRevB.46.2727).
- [29] G. R. Kneller, K. Hinsien, J. C. Smith, Fast evaluation of time correlation functions, Journal of Computational Chemistry 16 (7) (1995) 873–880. doi:[10.1002/jcc.540160709](https://doi.org/10.1002/jcc.540160709).
- [30] S. K. Lam, A. Pitrou, S. Seibert, Numba: A LLVM-based Python JIT compiler, in: Proceedings of the Second Workshop on the LLVM Compiler Infrastructure in HPC, Association for Computing Machinery, New York, NY, USA, 2015, pp. 1–6. doi:[10.1145/2833157.2833162](https://doi.org/10.1145/2833157.2833162).